Jira test case generator

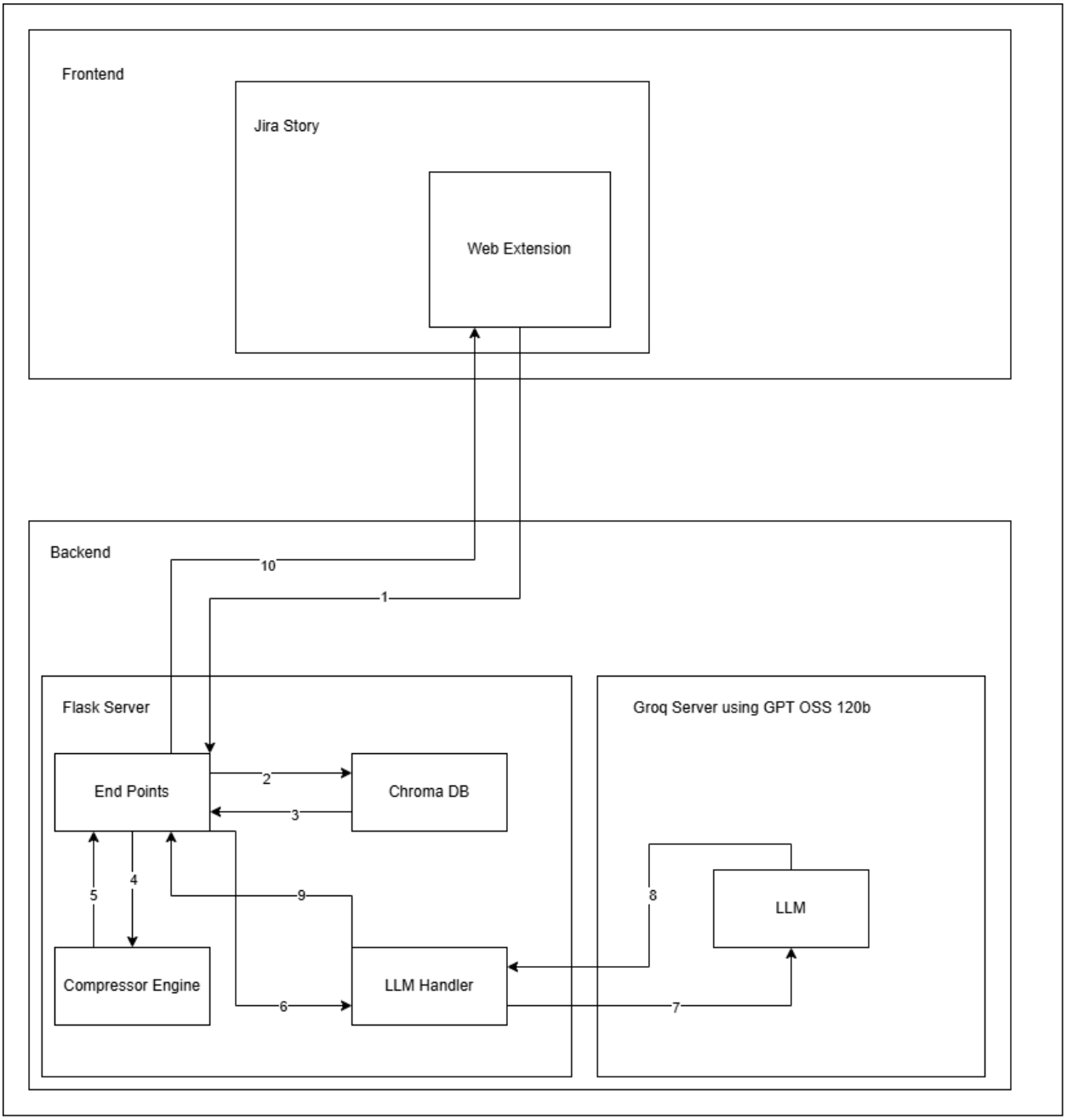
Objective: For a given story, user is able to generate test cases and add directly to Jira.

Motivation: Understanding a story completely, thinking of each test cases and include all the corner cases in a difficult task for testers. In case, if some corner cases are not found, it might return as bug in the future which delays the product development and delivery. The project aims to reduce the time taken for test case creation as well as cover wider test areas.

Advantages

- When testers have to write test cases, they mostly look into the story and its scenarios. But this tool looks into the related stories and relevant info from a vector database to give more context to the LLM while generating test cases. This helps to get deeper test cases to be made.
- The tool can create test cases in minutes which usually takes hours to do.
- The user can directly access this from the Jira story in browser via extension
- Manual changes can be made to the LLM response if needed.
- Can directly create test cases and link to the story in Jira.

Architecture Diagram



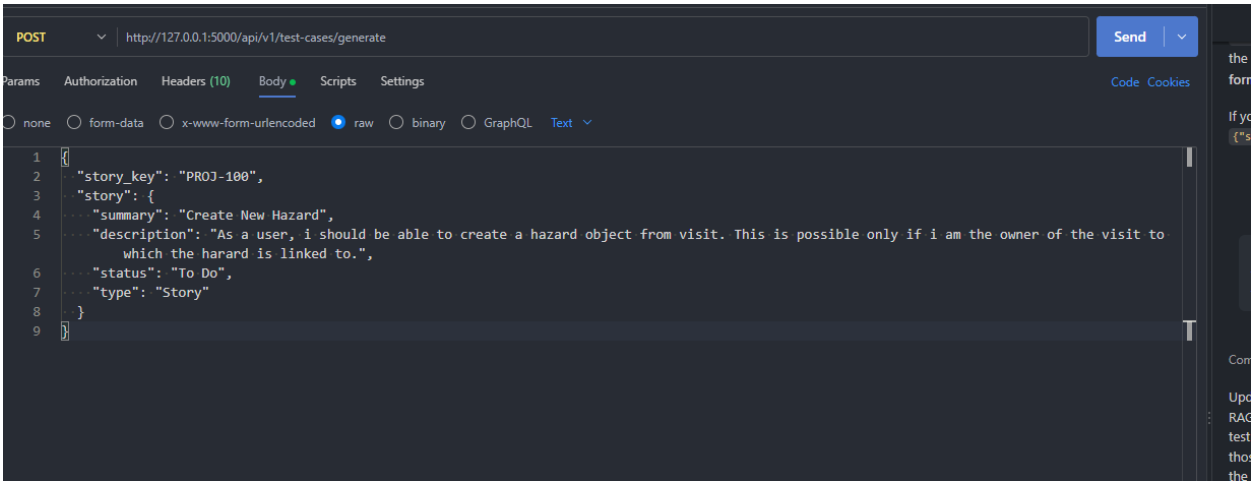
- The Web extension gets the description and other relevant info from the Webpage by accessing DOM.
- The details are converted to a Json format and are sent to the Flask server.

- The server uses rag to find the related and relevant content from chromaDB which will be populated everytime a new story is added to the DB.
- The message containing extra context is then sent to a compressor engine which removes unwanted tokens from the content.
- The filtered content is then sent to the LLM handler which connects to LLM Server via Groq API service.
- The model I used in this project is gpt-oss-120b
- The LLM return the result which has the test cases which is sent back the Front end
- The extension display the test cases in the UI where the user can edit the test case
- The user can also select which test case need to be save in to Jira.
- After selecting the required test cases, he has to click on Save to add those test cases to the story.

Note.....

The UI part is not fully implemented, only the base extension is made. To demo the project, I have used postman to send the request and JSON and response is also in JSON format.

Request



Response




```
{"data":{"story":{"summary":"Create New Hazard","description":"As a user, i should be able to create a hazard object from visit. This is possible only if i am the owner of the visit to which the harard is linked to.","status":"To Do","type":"Story","key":"PROJ-100"},"relevant_context":["{\"summary\": \"Unlock a locked account\", \"description\": \"As a user, I want to unlock my account after verifying my identity.\", \"status\": \"To Do\", \"type\": \"Story\", \"key\": \"PROJ-125\"}","{\"key\": \"PROJ-124\", \"summary\": \"Sign in with valid credentials\", \"description\": \"As a user, I want to sign in to my account.\", \"status\": \"To Do\", \"type\": \"Story\"}","{\"key\": \"PROJ-123\", \"summary\": \"Reset a forgotten password\", \"description\": \"As a user, I want to reset my password by email so I can regain access to my account.\", \"status\": \"In Progress\", \"type\": \"Story\"}"],"test_cases":[{"id":"GEN-001","story_key":"PROJ-100","title":"Owner creates hazard successfully","preconditions":["User is logged in","User is the owner of Visit ID 101"],"steps":["Navigate to Visit ID 101 page","Click 'Add Hazard' button","Enter valid hazard name and description","Click 'Save'"],"expected_result":"Hazard is created and linked to Visit ID 101, success message displayed"},{"id":"GEN-002","story_key":"PROJ-100","title":"Non-owner attempts to create hazard","preconditions":["User is logged in","User is NOT the owner of Visit ID 102"],"steps":["Navigate to Visit ID 102 page","Click 'Add Hazard' button"],"expected_result":"User receives an authorization error and cannot access the hazard creation form"},{"id":"GEN-003","story_key":"PROJ-100","title":"Owner submits hazard with missing required fields","preconditions":["User is logged in","User is the owner of Visit ID 103"],"steps":["Navigate to Visit ID 103 page","Click 'Add Hazard' button","Leave hazard name empty","Click 'Save'"],"expected_result":"Validation error indicating required fields are missing, hazard is not created"}],"llm_response":"{\"test_cases\":[{\"title\":\"Owner creates hazard successfully\",\"preconditions\":[\"User is logged in\",\"User is the owner of Visit ID 101\"],\"steps\":[\"Navigate to Visit ID 101 page\",\"Click 'Add Hazard' button\",\"Enter valid hazard name and description\",\"Click 'Save'\"],\"expected_result\":\"Hazard is created and linked to Visit ID 101, success message displayed\"},{\"title\":\"Non-owner attempts to create hazard\",\"preconditions\":[\"User is logged in\",\"User is NOT the owner of Visit ID 102\"],\"steps\":[\"Navigate to Visit ID 102 page\",\"Click 'Add Hazard' button\"],\"expected_result\":\"User receives an authorization error and cannot access the hazard creation form\"},{\"title\":\"Owner submits hazard with missing required fields\",\"preconditions\":[\"User is logged in\",\"User is the owner of Visit ID 103\"],\"steps\":[\"Navigate to Visit ID 103 page\",\"Click 'Add Hazard' button\",\"Leave hazard name empty\",\"Click 'Save'\"],\"expected_result\":\"Validation error indicating required fields are missing, hazard is not created\"}]}"}}
```

Github URL : https://github.com/A-L-O-K/Jira-Test-Case-Generator

Note : to run this in any system, they have to set up the venv and install the requirements.txt

To get Groq API, Visit *console.groq.com*